

Report Title:

Comparison and Analysis of DOM Manipulation in Vanilla JavaScript vs. jQuery

1. Introduction

- Briefly describe the project and its goals.
- Explain what will be covered in the report, including the focus on comparing and contrasting the two versions of the rendering and movement functions implemented using vanilla JavaScript and jQuery.

2. Background

- Provide a general explanation of what DOM manipulation is and why it's essential for web development.
- Describe both the Vanilla JS and JQuery approaches, highlighting key features of each and their typical use cases in web development.

3. Implementation Details

- **Vanilla JavaScript Version:**
 - **Code Explanation:** Provide a detailed explanation of how the rendering and chess piece movement was implemented using vanilla JavaScript, including code snippets.
 - **Challenges:** Discuss any challenges encountered while using vanilla JavaScript and how they were addressed.
- **jQuery Version:**
 - **Code Explanation:** Similarly, explain the implementation using jQuery, with relevant code examples.
 - **Challenges:** Discuss any challenges specific to using jQuery and how they were handled.

4. Comparison

- **Syntax:** Compare the syntax used in both implementations, noting which was more concise or easier to understand.
- **Performance:** Provide metrics regarding the performance of the two implementations. Use the Chrome dev tools for this, under the Performance tab. Provide screenshots for any data collection performed. Provide a **quantitative** analysis based on the metrics recorded, including **using data tables and charts** to illustrate and summarize your findings. **Be sure to consider both the**

initial page load times and piece movement times with an empty browser cache, and after JQuery has been cached (if applicable).

- **Ease of Use:** Analyze which approach was easier to work with and why, considering factors like code readability, maintainability, and learning curve.

6. Conclusion

- **Summary of Findings:** Summarize the key differences, advantages, and disadvantages of using vanilla JavaScript vs. JQuery for this specific task.
- **Personal Insights:** Share your personal insights or preferences you developed during this project.
- **Future Considerations:** Suggest when it might be more appropriate to use one approach over the other in future projects.

7. References

- **Citations:** Include references to any external resources or documentation consulted during the project.

8. Formatting and Submission Requirements

- - **File Format:** Submit the report in PDF format.

Note: There is a 5% per-day-late penalty between the posted due date / time, and the posted "Available Until" date / time. Submissions cannot be submitted after the "Available Until" date / time.

Introduction

In this project, I was responsible for constructing a chess program using HTML, CSS and Vanilla JavaScript (raw JavaScript without any libraries, plugins or frameworks) and JQuery. HTML was used to develop the frontend interface, CSS was used to provide aesthetics, and Javascript/JQuery was used to implement the logic of the chess game. For stability, I decided to clone the remote repository in my local machine rather than utilize codespaces. This strategy made it a lot easy for me to navigate through the codebase.

For this project, it was a required to have all the necessary chess pieces, however it not a requirement for each piece to have its required moves. The way the user moves each piece by utilizing a source text field (the coordinate of the piece the user wishes to move) and a destination text field (where the user wants the piece to move).

Background

For this project, I was required to utilize DOM (Document Object Model). Dom is an object that encapsulates all the HTML/CSS content and allow the programmer to perform computational operation utilizing HTML syntax.

For this project, it was a requirement to create two separate version of the chess program, one in Vanilla JavaScript and one in JQuery. Both languages contain the same logic however in JQuery, the syntax is more simplified and streamlined as opposed to Vanilla JavaScript. For example, when retrieving a tag in Vanilla JavaScript you would use the following syntax:

```
const table = document.createElement('table');
```

However, in JQuery you would use:

```
const $table = $('<table>');
```

Besides that, the logic and conceptualism of each language is the exact same.

Implementation Details

Because the Vanilla JavaScript and JQuery are syntactically similar and follow the same logic, building each version of the application (Vanilla Javascript and JQuery) was ultimately the same. For my Vanilla JavaScript version, I start building the HTML (chessboard.html same):

```
<!-- CSCI 620, Project 1 -->
<html>
  <head>
    <!-- Your JS file-->
    <script src="chessboard.js"></script>
    <!-- Your CSS file-->
    <link rel="stylesheet" href="chessboard.css">
    <!-- Latest compiled and minified Bootstrap CSS -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@3.3.7/dist/css/bootstrap.min.css"
integrity="sha384-BVYiISiFeK1dGmJRAkycuHAHRg32OmUcww7on3RYdg4Va+PmSTsz/K68vbdEjh4u"
crossorigin="anonymous">
    <!-- Optional Bootstrap theme -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@3.3.7/dist/css/bootstrap-theme.min.css"
integrity="sha384-rHyoN1iRsVXV4nD0JutlnGaslCJuC7uwjduW9SVrLvRYooPp2bWYgmgJQIXwl/Sp"
crossorigin="anonymous">
    <!-- Latest compiled and minified Bootstrap JavaScript -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@3.3.7/dist/js/bootstrap.min.js" integrity="sha384-
Tc5lQib027qvyjSMfHjOMaLkfuWVxZxUPnCJA7l2mCWNIpG9mGCD8wGNlCpD7Txa"
crossorigin="anonymous"></script>
    <!-- Font -->
    <link href="https://fonts.googleapis.com/css2?family=Fuzzy+Bubbles:wght@400;700&display=swap"
rel="stylesheet">
  </head>
  <body>

    <!-- Project Title-->
    <h1 class="Title"> CSCI 620: Project 1 - Chessboard</h1>

    <!-- Div used as a placeholder to render chessboard -->
    <div id="chessBoard"></div>

    <!-- Control settings used to move pieces -->
    <div class="control-settings">
      <!-- Source coordinate (piece you want to move) -->
      <input type="text" id="source" placeholder="g1" class="coordinates">

      <!-- Destination coordinate (where you wish for piece to move) -->
      <input type="text" id="destination" placeholder="f3" class="coordinates">

      <!-- Button used to move piece -->
      <button onclick="Move_Piece()" class="move-button">Move</button>

      <!-- Button used to reset board -->
```

```

    <button onclick="Reset()" class="reset-button">Reset</button>
  </div>

  <!-- Chess history-->
  <hr>
  <div class="chess-history">
    <h2>History of chess</h2>
    <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Suscipit, maiores non. Iste veniam quam reiciendis
    aliquid praesentium atque quidem, assumenda vero. Expedita incidunt fuga impedit, illo aut sit aliquam
    numquam!</p>
  </div>

  <!-- Description of Chess Pieces-->
  <div class="chess-pieces">
    <hr>
    <h2>Chess Piece Description</h2>
    <ul>
      <li><strong>Pawn:</strong> </li>
      <li><strong>Rook:</strong> </li>
      <li><strong>Bishop:</strong> </li>
      <li><strong>Knight:</strong> </li>
      <li><strong>Queen:</strong> </li>
      <li><strong>King:</strong> </li>
    </ul>
  </div>

  <div class="about-author">
    <hr>
    <h2>About Author</h2>
    <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Suscipit, maiores non. Iste veniam quam reiciendis
    aliquid praesentium atque quidem, assumenda vero. Expedita incidunt fuga impedit, illo aut sit aliquam
    numquam!</p>
  </div>

</body>
</html>

```

From there, I transitioned to chessboard.css:

```

body
{

```

```
padding: 40px;
background: linear-gradient(
  rgb(230, 230, 230), rgb(255, 234, 217))
}

table
{
margin: auto;
border: 1px solid black;
}
td {
background: tan;
width: 60px;
height: 60px;
border: 1px solid black;
font-size: 60px;
text-align: center;
}

th {
background: tan;
width: 60px;
height: 60px;
border: 1px solid black;
text-align: center;
font-weight: bold;
font-size: 20px;
}

/* Alternating square colors: odd cells in odd rows, even cells in even rows = chocolate */
tr:nth-child(odd)>td:nth-child(odd) {
background: chocolate;
}

tr:nth-child(even)>td:nth-child(even) {
background: chocolate;
}

.title
{
text-align: center;
font-family: "Fuzzy Bubbles", sans-serif;
font-size: 30px;
}

.chess-history,
.chess-pieces,
```

```
.about-author
{
  margin-top: 30px;
  max-width: 800px;
  margin-left: auto;
  margin-right: auto;
}

.control-settings
{
  margin-top: 30px;
  text-align: center;
}

.control-settings input {
  margin: 0 10px;
}

.control-settings button
{
  margin: 0 10px;
  background-color: rgb(255, 202, 149);
  border: 0px;
  border-radius: 5px;
  width: 90px;
  font-family: "Fuzzy Bubbles", sans-serif;
  font-size: 14px;
}

button:hover
{
  color: white;
}

.control-settings .coordinates {
  display: inline-block;
  width: 100px;
  vertical-align: middle;
}
```

Lastly, I worked on chessboard.js which contained the logic of the application:

```
/*
  Create your javascript functions for Project 1 here, using vanilla javascript syntax to manipulate the DOM...
*/

// Declare json object of each chess piece
// Key = piece, value = color
const CHESS_PIECES =
{
  'whiteKing': '♔',
  'blackKing': '♚',
  'whiteQueen': '♕',
  'blackQueen': '♛',
  'whiteRook': '♖',
  'blackRook': '♜',
  'whiteBishop': '♗',
  'blackBishop': '♝',
  'whiteKnight': '♘',
  'blackKnight': '♞',
  'whitePawn': '♙',
  'blackPawn': '♟',
  'empty': '&nbsp;';
};

// Declare JSON object of board positioning in beginning of match
const POSITION =
{
  'a8': CHESS_PIECES.blackRook, 'b8': CHESS_PIECES.blackKnight, 'c8': CHESS_PIECES.blackBishop, 'd8':
CHESS_PIECES.blackQueen,
  'e8': CHESS_PIECES.blackKing, 'f8': CHESS_PIECES.blackBishop, 'g8': CHESS_PIECES.blackKnight, 'h8':
CHESS_PIECES.blackRook,
  'a7': CHESS_PIECES.blackPawn, 'b7': CHESS_PIECES.blackPawn, 'c7': CHESS_PIECES.blackPawn, 'd7':
CHESS_PIECES.blackPawn,
  'e7': CHESS_PIECES.blackPawn, 'f7': CHESS_PIECES.blackPawn, 'g7': CHESS_PIECES.blackPawn, 'h7':
CHESS_PIECES.blackPawn,
  'a6': CHESS_PIECES.empty, 'b6': CHESS_PIECES.empty, 'c6': CHESS_PIECES.empty, 'd6':
CHESS_PIECES.empty,
  'e6': CHESS_PIECES.empty, 'f6': CHESS_PIECES.empty, 'g6': CHESS_PIECES.empty, 'h6':
CHESS_PIECES.empty,
  'a5': CHESS_PIECES.empty, 'b5': CHESS_PIECES.empty, 'c5': CHESS_PIECES.empty, 'd5':
CHESS_PIECES.empty,
  'e5': CHESS_PIECES.empty, 'f5': CHESS_PIECES.empty, 'g5': CHESS_PIECES.empty, 'h5':
CHESS_PIECES.empty,
  'a4': CHESS_PIECES.empty, 'b4': CHESS_PIECES.empty, 'c4': CHESS_PIECES.empty, 'd4':
CHESS_PIECES.empty,
```



```

    'e4': CHESS_PIECES.empty, 'f4': CHESS_PIECES.empty, 'g4': CHESS_PIECES.empty, 'h4':
CHESS_PIECES.empty,
    'a3': CHESS_PIECES.empty, 'b3': CHESS_PIECES.empty, 'c3': CHESS_PIECES.empty, 'd3':
CHESS_PIECES.empty,
    'e3': CHESS_PIECES.empty, 'f3': CHESS_PIECES.empty, 'g3': CHESS_PIECES.empty, 'h3':
CHESS_PIECES.empty,
    'a2': CHESS_PIECES.whitePawn, 'b2': CHESS_PIECES.whitePawn, 'c2': CHESS_PIECES.whitePawn, 'd2':
CHESS_PIECES.whitePawn,
    'e2': CHESS_PIECES.whitePawn, 'f2': CHESS_PIECES.whitePawn, 'g2': CHESS_PIECES.whitePawn, 'h2':
CHESS_PIECES.whitePawn,
    'a1': CHESS_PIECES.whiteRook, 'b1': CHESS_PIECES.whiteKnight, 'c1': CHESS_PIECES.whiteBishop, 'd1':
CHESS_PIECES.whiteQueen,
    'e1': CHESS_PIECES.whiteKing, 'f1': CHESS_PIECES.whiteBishop, 'g1': CHESS_PIECES.whiteKnight, 'h1':
CHESS_PIECES.whiteRook
};

```

```

// Declare chessboard dataset

```

```

let chessBoard = {};

```

```

// -----

```

```

// Generate design of chessboard

```

```

function Render_Board() {

```

```

    // Fetch chessBoard div from DOM and store it in variable (chessBoardDiv)

```

```

    const board = document.getElementById('chessBoard');

```

```

    // Produce table variable

```

```

    const table = document.createElement('table');

```

```

    // Insert 8 rows (from 8 to 1)

```

```

    for (let row = 8; row >= 1; row--) {

```

```

        // Declare tr tag variable

```

```

        const tr = document.createElement('tr');

```

```

        // Declare th element

```

```

        const th = document.createElement('th')

```

```

        // Assign header value (1-8) displayed vertically on chessboard

```

```

        th.textContent = row;

```

```

        // Store head value

```

```

        tr.appendChild(th);

```

```

        // Assign coordinates for each square on chessboard

```

```

        for (let column = 0; column < 8; column += 1)

```

```

        {

```

```

            // Declare td element

```

```

const td = document.createElement('td');

// Assign id (coordinate) of each individual square --> String.fromCharCode(97) = 'a' + row #
td.id = String.fromCharCode(97 + column) + row;

// Store data value
tr.appendChild(td);
}

// Store row value in table
table.appendChild(tr);
}

// Insert column letters at bottom of chessboard (a-h)
const row = document.createElement('tr');
const emptyCorner = document.createElement('th');
emptyCorner.textContent = "";
row.appendChild(emptyCorner);

// Traverse through column numbers
for (let column = 0; column < 8; column += 1) {
  const th = document.createElement('th');
  th.textContent = String.fromCharCode(97 + column);
  row.appendChild(th);
}

// Add row to table
table.appendChild(row);

// Render chessboard
board.appendChild(table);
}

// -----

// Render logic of each chess board piece
function Render_Pieces() {
  // Reset chessboard dataset
  chessBoard = {};

  // Key = position of board, value = unicode character of chess piece
  for (let p in POSITION)
    chessBoard[p] = POSITION[p];

  // Display chess pieces by traversing through each square on chessboard
  for (let coordinate in chessBoard) {

```

```

// Retrieve id of each coordinate
const id = document.getElementById(coordinate);

// If id exists, assign chess piece to each coordinate
if (id)
    id.innerHTML = chessBoard[coordinate];
}

}

// -----

// Logic used to move each individual chess piece
function Move_Piece()
{
    // Retrieve input value of source and destination. Convert strings to lower case and trims extra spaces
    const source = document.getElementById('source').value.toLowerCase().trim();
    const destination = document.getElementById('destination').value.toLowerCase().trim();

    // Input validation - Determine if user enter correct input
    if (!/^[a-h][1-8]$/.test(source) || !/^[a-h][1-8]$/.test(destination)) {
        alert('Invalid coordinate - please use correct format. Ex - "e2" or "a1"');
        return;
    }

    // Retrieve id for source cell
    const sourceCell = document.getElementById(source);

    // Retrieve id for destination cell
    const destinationCell = document.getElementById(destination);

    // Verify if cell is valid
    if (!sourceCell || !destinationCell) {
        alert('Error - invalid coordinate.\n\nPlease re-enter');
        return;
    }

    // Assign unicode value of piece
    const piece = chessBoard[source] || sourceCell.innerHTML;

    // Assign value
    destinationCell.innerHTML = piece;
    chessBoard[destination] = piece;

    // Clear source square
    sourceCell.innerHTML = CHESS_PIECES.empty;
    chessBoard[source] = CHESS_PIECES.empty;

```

```

// Clear text field data
document.getElementById('source').value = "";
document.getElementById('destination').value = "";
}

// Logic used reset game
function Reset() {
    // Re-render pieces
    Render_Pieces();

    // Clear text field data
    document.getElementById('source').value = "";
    document.getElementById('destination').value = "";
}

// -----

// Generate board once page loads
window.onload = function () {
    // Generate board
    Render_Board();

    // Generate chess pieces
    Render_Pieces();
}

// -----

```

The challenges I encountered for the Vanilla JavaScript version included: incorrect logic in for loop, typo in `String.fromCharCode(97+column)`, typo in function declaration, typo in function names, typo in variable names and incorrect logic used on move and reset buttons. Therefore, my solution was to utilize the debugging tool provided by Visual Studio Code used in my local machine. I also included several debug prints allow me to determine whether the syntax was properly operating. These strategies ultimately helped me fix all the errors and bugs present within the application.

For the JQuery version of my application, I followed the same strategy. I copied and pasted the .html and .css files since they are the exact same. I have posted them below:

```

<!-- CSCI 620, Project 1 -->

<html>
<head>

```

```

    <!-- JQuery-->
    <script src="https://code.jquery.com/jquery-3.7.1.min.js" integrity="sha256-
/JqT3SQfawRcv/BIHPThkBs00EvtFFmqPF/ly/Cxo=" crossorigin="anonymous"></script>
    <!-- Your JS file-->
    <script src="chessboard.js"></script>
    <!-- Your CSS file-->
    <link rel="stylesheet" href="chessboard.css">
    <!-- Latest compiled and minified Bootstrap CSS -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@3.3.7/dist/css/bootstrap.min.css"
integrity="sha384-BVYiISiFeK1dGmJRAkycuHAHRg32OmUcww7on3RYdg4Va+PmSTsz/K68vbdEjh4u"
crossorigin="anonymous">
    <!-- Optional Bootstrap theme -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@3.3.7/dist/css/bootstrap-theme.min.css"
integrity="sha384-rHyoN1iRsVXV4nD0JutlnGaslCJuC7uwjduW9SVrLvRYooPp2bWYgmgJQIXwl/Sp"
crossorigin="anonymous">
    <!-- Latest compiled and minified Bootstrap JavaScript -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@3.3.7/dist/js/bootstrap.min.js" integrity="sha384-
Tc5lQib027qvyjSMfHjOMaLkfuWVxZxUPnCJA7I2mCWNlpG9mGCD8wGNlcPD7Txa"
crossorigin="anonymous"></script>
    <!-- Font -->
    <link href="https://fonts.googleapis.com/css2?family=Fuzzy+Bubbles:wght@400;700&display=swap"
rel="stylesheet">
</head>
<body>

    <!-- Project Title-->
    <h1 class="Title"> CSCI 620: Project 1 - Chessboard</h1>

    <!-- Div used as a placeholder to render chessboard -->
    <div id="chessBoard"></div>

    <!-- Control settings used to move pieces -->
    <div class="control-settings">
        <!-- Source coordinate (piece you want to move) -->
        <input type="text" id="source" placeholder="g1" class="coordinates">

        <!-- Destination coordinate (where you wish for piece to move) -->
        <input type="text" id="destination" placeholder="f3" class="coordinates">

        <!-- Button used to move piece -->
        <button onclick="Move_Piece()" class="move-button">Move</button>

        <!-- Button used to reset board -->
        <button onclick="Reset()" class="reset-button">Reset</button>
    </div>

    <!-- Chess history-->
    <hr>

```

```
<div class="chess-history">
  <h2>History of chess</h2>
  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Suscipit, maiores non. Iste veniam quam reiciendis
aliquid praesentium atque quidem, assumenda vero. Expedita incidunt fuga impedit, illo aut sit aliquam
numquam!</p>
</div>

<!-- Description of Chess Pieces-->
<div class="chess-pieces">
  <hr>
  <h2>Chess Piece Description</h2>
  <ul>
    <li><strong>Pawn:</strong> </li>
    <li><strong>Rook:</strong> </li>
    <li><strong>Bishop:</strong> </li>
    <li><strong>Knight:</strong> </li>
    <li><strong>Queen:</strong> </li>
    <li><strong>King:</strong> </li>
  </ul>
</div>

<div class="about-author">
  <hr>
  <h2>About Author</h2>
  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Suscipit, maiores non. Iste veniam quam reiciendis
aliquid praesentium atque quidem, assumenda vero. Expedita incidunt fuga impedit, illo aut sit aliquam
numquam!</p>
</div>

</body>
</html>

body
{
padding: 40px;
background: linear-gradient(
  rgb(230, 230, 230), rgb(255, 234, 217))
}

table
{
margin: auto;
border: 1px solid black;
}

td
{
background: tan;
```

```

width: 60px;
height: 60px;
border: 1px solid black;
font-size: 60px;
text-align: center;
}

th
{
background: tan;
width: 60px;
height: 60px;
border: 1px solid black;
text-align: center;
font-weight: bold;
font-size: 20px;
}

tr:nth-child(odd)>td:nth-child(odd)
{
background: chocolate;
}

tr:nth-child(even)>td:nth-child(even)
{
background: chocolate;
}

.title
{
text-align: center;
font-family: "Fuzzy Bubbles", sans-serif;
font-size: 30px;
}

.chess-history,
.chess-pieces,
.about-author
{
margin-top: 30px;
max-width: 800px;
margin-left: auto;
margin-right: auto;
}

.control-settings
{

```

```
margin-top: 30px;
text-align: center;
}

.control-settings input {
margin: 0 10px;
}

.control-settings button
{
margin: 0 10px;
background-color: rgb(255, 202, 149);
border: 0px;
border-radius: 5px;
width: 90px;
font-family: "Fuzzy Bubbles", sans-serif;
font-size: 14px;
}

button:hover
{
color: white;
}

.control-settings .coordinates {
display: inline-block;
width: 100px;
vertical-align: middle;
}
```


From there, I began working on the logic used for JQuery:

```
// Declare json object of each chess piece

// Key = piece, value = color
const CHESS_PIECES =
{
  'whiteKing': '♔',
  'blackKing': '♚',
  'whiteQueen': '♕',
  'blackQueen': '♛',
  'whiteRook': '♖',
  'blackRook': '♜',
  'whiteBishop': '♗',
  'blackBishop': '♝',
  'whiteKnight': '♘',
  'blackKnight': '♞',
  'whitePawn': '♙',
  'blackPawn': '♟',
  'empty': '&nbsp;'
};

// Declare JSON object of board positioning in beginning of match
const POSITION =
{
  'a8': CHESS_PIECES.blackRook, 'b8': CHESS_PIECES.blackKnight, 'c8': CHESS_PIECES.blackBishop, 'd8':
CHESS_PIECES.blackQueen,
  'e8': CHESS_PIECES.blackKing, 'f8': CHESS_PIECES.blackBishop, 'g8': CHESS_PIECES.blackKnight, 'h8':
CHESS_PIECES.blackRook,
  'a7': CHESS_PIECES.blackPawn, 'b7': CHESS_PIECES.blackPawn, 'c7': CHESS_PIECES.blackPawn, 'd7':
CHESS_PIECES.blackPawn,
  'e7': CHESS_PIECES.blackPawn, 'f7': CHESS_PIECES.blackPawn, 'g7': CHESS_PIECES.blackPawn, 'h7':
CHESS_PIECES.blackPawn,
  'a6': CHESS_PIECES.empty, 'b6': CHESS_PIECES.empty, 'c6': CHESS_PIECES.empty, 'd6':
CHESS_PIECES.empty,
  'e6': CHESS_PIECES.empty, 'f6': CHESS_PIECES.empty, 'g6': CHESS_PIECES.empty, 'h6':
CHESS_PIECES.empty,
  'a5': CHESS_PIECES.empty, 'b5': CHESS_PIECES.empty, 'c5': CHESS_PIECES.empty, 'd5':
CHESS_PIECES.empty,
  'e5': CHESS_PIECES.empty, 'f5': CHESS_PIECES.empty, 'g5': CHESS_PIECES.empty, 'h5':
CHESS_PIECES.empty,
  'a4': CHESS_PIECES.empty, 'b4': CHESS_PIECES.empty, 'c4': CHESS_PIECES.empty, 'd4':
CHESS_PIECES.empty,
```

```

    'e4': CHESS_PIECES.empty, 'f4': CHESS_PIECES.empty, 'g4': CHESS_PIECES.empty, 'h4':
CHESS_PIECES.empty,
    'a3': CHESS_PIECES.empty, 'b3': CHESS_PIECES.empty, 'c3': CHESS_PIECES.empty, 'd3':
CHESS_PIECES.empty,
    'e3': CHESS_PIECES.empty, 'f3': CHESS_PIECES.empty, 'g3': CHESS_PIECES.empty, 'h3':
CHESS_PIECES.empty,
    'a2': CHESS_PIECES.whitePawn, 'b2': CHESS_PIECES.whitePawn, 'c2': CHESS_PIECES.whitePawn, 'd2':
CHESS_PIECES.whitePawn,
    'e2': CHESS_PIECES.whitePawn, 'f2': CHESS_PIECES.whitePawn, 'g2': CHESS_PIECES.whitePawn, 'h2':
CHESS_PIECES.whitePawn,
    'a1': CHESS_PIECES.whiteRook, 'b1': CHESS_PIECES.whiteKnight, 'c1': CHESS_PIECES.whiteBishop, 'd1':
CHESS_PIECES.whiteQueen,
    'e1': CHESS_PIECES.whiteKing, 'f1': CHESS_PIECES.whiteBishop, 'g1': CHESS_PIECES.whiteKnight, 'h1':
CHESS_PIECES.whiteRook
  };

```

```

// Declare chessboard dataset

```

```

let chessBoard = {};

```

```

// -----

```

```

// Generate design of chessboard

```

```

function Render_Board() {

```

```

  // Fetch chessboard container div from DOM and store it in variable using jQuery
  const $board = $('#chessBoard');

```

```

  // Produce table variable using jQuery
  const $table = $('<table>');

```

```

  // Insert 8 rows (from 8 to 1)

```

```

  for (let row = 8; row >= 1; row--) {

```

```

    // Declare tr tag variable using jQuery
    const $tr = $('<tr>');

```

```

    // Declare th element using jQuery

```

```

    const $th = $('<th>');

```

```

    // Assign header value (1-8) displayed vertically on chessboard

```

```

    $th.text(row);

```

```

    // Store head value

```

```

    $tr.append($th);

```

```

    // Assign coordinates for each square on chessboard

```

```

    for (let column = 0; column < 8; column += 1) {

```

```

      // Declare td element using jQuery

```

```

      const $td = $('<td>');

```

```

    // Assign id (coordinate) of each individual square --> String.fromCharCode(97) = 'a' + row #
    $td.attr('id', String.fromCharCode(97 + column) + row);

    // Store data value
    $tr.append($td);
}

// Store row value in table
$table.append($tr);

}

// Insert column letters at bottom of chessboard (a-h)
const $row = $('<tr>');
const $emptyCorner = $('<th>');
$emptyCorner.text("");
$row.append($emptyCorner);

// Traverse through column numbers
for (let column = 0; column < 8; column += 1) {
    const $th = $('<th>');
    $th.text(String.fromCharCode(97 + column));
    $row.append($th);
}

// Add row to table
$table.append($row);

// Render chessboard
$board.append($table);

}

// -----

// Render logic of each chess board piece
function Render_Pieces()
{
    chessBoard = {};
    for (let p in POSITION)
        chessBoard[p] = POSITION[p];
    for (let coordinate in chessBoard) {
        const $cell = $('<#>' + coordinate);
        if ($cell.length)
            $cell.html(chessBoard[coordinate]);
    }
}

```

```

// -----

// Logic used to move each individual chess piece
function Move_Piece() {
    // Retrieve input value of source and destination. Convert strings to lower case and trims extra spaces (using jQuery .val())
    const source = $('#source').val().toLowerCase().trim();
    const destination = $('#destination').val().toLowerCase().trim();

    // Input validation - Determine if user enter correct input
    if (!/^[a-h][1-8]$/.test(source) || /^[a-h][1-8]$/.test(destination)) {
        alert('Invalid coordinate - please use correct format. Ex - "e2" or "a1"');
        return;
    }

    // Retrieve id for source cell using jQuery
    const $sourceCell = $('# ' + source);

    // Retrieve id for destination cell using jQuery
    const $destinationCell = $('# ' + destination);

    // Verify if cell is valid
    if (!$sourceCell.length || !$destinationCell.length) {
        alert('Error - invalid coordinate.\n\nPlease re-enter');
        return;
    }

    // Assign unicode value of piece
    const piece = chessBoard[source] || $sourceCell.html();

    // Assign value
    $destinationCell.html(piece);
    chessBoard[destination] = piece;

    // Clear source square
    $sourceCell.html(CHESS_PIECES.empty);
    chessBoard[source] = CHESS_PIECES.empty;

    // Clear text field data (using jQuery .val())
    $('#source').val('');
    $('#destination').val('');
}

// Logic used reset game
function Reset()
{
    // Re-render pieces

```

```
Render_Pieces();

// Clear text field data
$('#source').val("");
$('#destination').val("");
}

// Generate board once page loads
$(document).ready(function () {
    // Generate board
    Render_Board();

    // Generate chess pieces
    Render_Pieces();
});
```

One strategy I used was to review the JQuery documentation (<https://api.jquery.com/>) which provided me an overview of the different commands that could be utilized for the chessboard app.

The challenges I encountered were very similar to the errors that occurred while I was working on the Vanilla JavaScript version such as typos and incorrect syntax errors. Although, because I had already completed the Vanilla JavaScript version, navigating through the application using JQuery was a lot more convenient.

Comparing Vanilla JavaScript to JQuery side by side, the only difference is the syntax used. In Vanilla JavaScript, the syntax is more verbose, while in JQuery the syntax more streamline and simplified. However, personally I prefer Vanilla JavaScript because the syntax is more detailed and self explanatory compared to JQuery. Even though I do see JQuery value, I personally prefer syntax that provides more detail and practicality, even if that requires more syntax. Therefore, I prefer using Vanilla JavaScript.

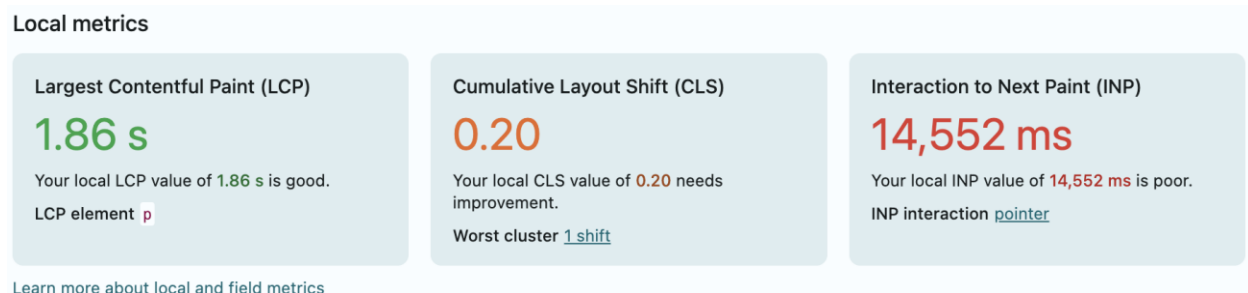
Comparison

I decided to launch my Vanilla JavaScript version and my JQuery version at the same time. From there, I went to *inspect -> performance* and retrieved the following metrics:

- *Largest Contentful Paint (LCP)* – The time it takes for the largest content to appear on a webpage. Anything under 2.5 second is considered good.
- *Cumulative Layout Shift (CLS)* – How much visible content shifts or moves around during rendering. Anything under 0.1 is considered good.
- *Interaction to Next Paint (INP)* – The time it takes for a user to interact with a component. Anything under 200 milliseconds (ms) is considered good.

Below I have provided the metrics:

Vanilla JavaScript:



JQuery:



According to the data, JQuery runs better performance than Vanilla JavaScript. However, the CLS and INP for both versions have room for improvement, which I plan to investigate in the future.

Conclusion

Overall, I personally preferred using Vanilla JavaScript over JQuery. I think I am more comfortable using JavaScript considering the fact that I have used C++ for 6 years and Selenium Java for 3 years, which seems to align more with those languages than JQuery. However, Vanilla JavaScript syntactically makes more sense than JQuery in my opinion. I think JQuery is a valuable library and should be used for time restraining purposes over JavaScript. But in my opinion, if you prioritize practicality over speed, then I would highly recommend Vanilla JavaScript over JQuery. Therefore, I personally prefer Vanilla JavaScript over JQuery. In the future, I plan to continue to analyze this project further and attempt to developed the logic used to regulate the move of each piece. I also wanted to work on adopting more advanced features and utilities such as bootstrap.

References

Why lab and field data can be different (and what to do about it)

https://web.dev/articles/lab-and-field-data-differences?utm_source=devtools&utm_campaign=stable#lab_data_versus_field_data

JQuery Documentation

<https://api.jquery.com/>

History of Chess

https://en.wikipedia.org/wiki/History_of_chess

